Master of Science in Computer Science
June 2021

Functional Decomposition Techniques and Their Impact on Performance, Scalability and Maintainability

Jonne van Dreven

Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Computing at Blekinge Institute of Technology in partial fulfilment of the requirements for the degree of Master of Science in Computer Science. The thesis is equivalent to 20 weeks of full time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:
Author(s):
Jonne van Dreven
E-mail: jova19@student.bth.se

University advisor:
Dr.-Tech. Ahmad Nauman Ghazi
Department of Software Engineering

Faculty of Computing
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Context The last decade shows many solution proposals of functional decomposition techniques to aid in developing microservice architectures. While some solutions may work, it is uncertain what the effects are on quantitative, measurable metrics; thus, the proposals require validation.
Objective The study measures the effects of various functional decomposition techniques on performance, scalability, and maintainability. Furthermore, the study will compare the treatments in order to find whether a statistical significance exists.
Method The study uses a controlled experiment containing three functional decomposition techniques – Event Storming, Actor/Action, and Service Cutter – applied on the same use case. The use case follows the CoCoMe framework, which forms the basis of the experiment.
Results Each treatment shows similar behavior while presenting different architectural designs. The study found no statistical significance for performance, scalability, and maintainability.
Conclusion Evidence suggests that the convenience of an approach might be more important than the resulting architecture since they will likely lead to the same outcome. If performance issues arise, it would likely be due to the microservices architecture and not the functional decomposition technique; therefore, the microservices architecture might not equally benefit any situation or corporation. Furthermore, the study found that service granularity might not be as relevant as some studies claim it to be, and other factors could be more crucial.

Keywords: Software Architecture, Cloud Computing, Microservices, Functional Decomposition.

Acknowledgments

I would like to show my gratitude towards my supervisor Dr.-Tech. Ahmad Nauman Ghazi for his trust, support, and inspiring me to reach further. Because of you, I was able to push my limits which I could not have done alone. Furthermore, I would like to thank Blekinge Institute of Technology for teaching me the foundations of research and all the knowledge I needed to do high-quality research.

Contents

Abstract **i**

Acknowledgments **iii**

1 Introduction **1**
1.1 Research Question 2
1.2 Hypotheses 2
1.3 Service Cutter 2
1.4 Actor/Actions Approach 3
1.5 Event Storming 3
1.6 Overview 4

2 Related Work **5**

3 Method **9**
3.1 Systematic Literature Review 9
3.2 Controlled Experiment 10
3.2.1 Validity Threats 10
3.2.2 Internal Validity 10
3.2.3 External Validity 11
3.2.4 Independent Variables 11
3.2.5 Dependent Variables 12
3.2.6 Data Gathering 12
3.2.7 Data Analysis 13
3.3 System 14
3.3.1 Implementation 14
3.3.2 Deployment 14
3.3.3 Purpose 14
3.3.4 Actors 14
3.3.5 Requirements 15
3.4 Architectural Overview 15
3.4.1 Monolith 15
3.4.2 Event Storming 19
3.4.3 Actor/Action 19
3.4.4 Service Cutter 20

4 Results and Discussion **23**
4.1 Results . . . 23
4.1.1 Performance . . . 23
4.1.2 Scalability . . . 24
4.1.3 Maintainability . . . 27
4.2 Discussion . . . 27
4.2.1 Limitations . . . 28
4.2.2 Lessons Learned . . . 29

5 Conclusions and Future Work **31**
5.1 Conclusion . . . 31
5.2 Future Work . . . 31

A Supplemental Information **37**

Chapter 1

Introduction

In 2020 Netflix had to reduce the bitrate to prevent the internet from slowing down. While the internet was facing difficulties, Netflix had no trouble handling the massive influx of load on their scalable microservices architecture. In the last decade, microservices changed to a well-established distributed software architecture capable of efficiently scaling and improving software development.

Microservices are small, independently deployable programs that together form a distributed system. They communicate with each other over a network, often through REST calls, although event-driven microservices are starting to arise. To achieve independent deployment, explicit service boundaries and well-defined interfaces are critical. Microservices make it possible to deploy a single service into production without the need to deploy anything else. Microservices make it possible to combine different programming languages, deployment platforms, and technologies. Microservices bring flexibility, scale quickly to serve massive amounts of traffic, and introduce more options to solve future problems. However, they are also potentially challenging to build.

Lewis and Fowler [16] mention such a problem, suggesting the name might emphasize size too much. However, size is a vital part of software development since it is often the root cause of problems in monoliths, i.e., size often results in unmaintainable and unsustainable software architectures. Developers are hesitant to start building microservices since no clear definitions exist. It is therefore essential to discuss size and define the service boundaries of a microservice.

While a systematically created collection of many small services is a microservices architecture, it is essential to understand the effects on measurable quantitative quality attributes. A significant factor in building sustainable software is building repetitive deterministic architectures with known consequences. Repetition of past events makes it possible to reason about repeating situations. Some functional decomposition techniques might present a solution and force developers to systematically designing software to transform a monolith or built a system from the start.

Functional decomposition is an analysis method to separate a complex process to examine individual components. The aim of the study is to compare various functional decomposition techniques and their effects on performance, scalability, and maintainability. In specific three functional decomposition techniques: Event Storming; Actor/Action approach; Service Cutter. Functional decomposition techniques could oppose a solution to algorithmically build software architectures making it easier to repeating proven practices. Ultimately, making it easier to create sustainable software products. Functional decomposition is a clearly defined problem within the

literature, and many studies have suggested several solutions. However, most studies were studies comparing resulting software diagrams, i.e., it is difficult to understand the effects based on solely diagrams. Therefore it might be necessary to build a model of an approach to measure quality attributes and using statistical techniques to form conclusions.

1.1 Research Question

The study is concerned with the following research question:

Research Question 1 (RQ1). *What are the effects of Service Cutter, Actor/Action approach, and Event Storming on performance, scalability, and maintainability when developing a microservices architecture?*

1.2 Hypotheses

The study will test three hypotheses:

Hypothesis 1 (H1). *The functional decomposition techniques do not show a statistical significance in performance.*

Hypothesis 2 (H2). *The functional decomposition techniques do not show a statistical significance in scalability.*

Hypothesis 3 (H3). *The functional decomposition techniques do not show a statistical significance in maintainability.*

1.3 Service Cutter

Service Cutter [9] is a tool that can transform functional requirements (text) into a microservices architectural design (diagram) using various algorithms. A slight drawback of the service cutter is that it needs a domain model in an Entity Relation Diagram (ERD). While this is not the issue itself, the ERD is a textual representation of possible microservices. Hence constructing such a diagram might hold the same issues as designing a microservices architecture. Below outlines several definitions which relate to Service Cutter.

Nanoentitites: refers to building blocks of possible services or known as service cuts. Service Cutter identifies three types of nanoentities.

Data: A service has ownership of arbitrary data. Data is not limited to databases since it can also refer cache storage.

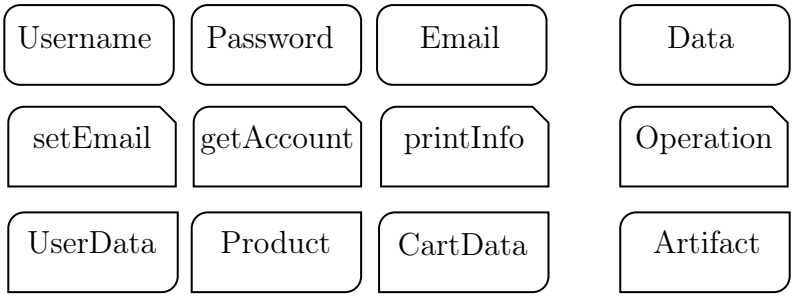

Figure 1.1: Visualisation of Nanoentities

Operations: Operations include business rules and logic. The operations often interact with the data but are not limited to. Stateless services coexists in a microservices ecosystem.

Artifacts: A collection of data or operations presented into a specific format e.g. a business report.

The three resource types mentioned above are generalized in what is called nanoentities. Image 1.1 displays a graphical representation of nanoentities.

1.4 Actor/Actions Approach

The actor/action approach [20] originating from the Rational Unified Process is an approach to map requirements to components. Typically actors who perform activities are identified, i.e., it is an approach to identify typical users and the things they do within the system. Developers use the actor/action approach when a significant upfront design is required. Once all actors and activities are discovered, the developer designs the systems around the actors and activities.

1.5 Event Storming

Event storming [20] is a component discovery technique from Domain-Driven-Design which developers use to identify microservices. The technique focuses on the communication between services. The first step is to determine possible events that occur within a system. The second step is to build services around the discovered events to reduce communication overhead.

API calls are expensive due to their overhead; hence by reducing them, microservices can gain performance. Therefore event storming is focused on discovering domain events to construct a software architecture and reduce communication.

Domain Event: A collection of data or operations presented into a specific format, e.g., a business report.

Domain Expert: A person with specific knowledge about a particular area.

1.6 Overview

The study is structured as follows. Chapter 2 Related Work presents a literature review, which discusses relevant work from other research, and places it into perspective in relation to the current study. Chapter 3 Method presents a throughout explanation on the study methodology and any relevant practices to study the topic. Chapter 4 Results and Discussion first presents the study results, which after presents an in-depth discussion on the results, possible limitations and lessons learned. Finally, chapter 5 Conclusion presents a summary of the study and points out potential future directions.

Chapter 2

Related Work

Rodgers first introduced the term microservices during a conference in 2005, indicating the industry is ahead. However, researchers are starting to reduce the gaps between industry and academia. A study by Francesco et al. [7] shows that in 2015 a significant number of microservices-related publications is starting to appear, which doubled in 2016. The majority of research papers are solution proposals which might suggest the microservices research topic is still in its infancy. The following chapter presents a literature review that is mainly concerned with the design and implementation of microservices and any essential subjects related to functional decomposition.

Many secondary studies are focusing on the architectural challenges and outline research challenges and directions. A study by Alshyqayran et al. [1] is concerned with identifying architectural challenges and implementations. In specific, the study focuses on diagrams, views, and quality attributes. Their study found that the literature mainly mentions scalability, independence, maintainability, and deployment as characteristics of a microservices architecture. A study by Jamshidi et al. [13] presents an overview of software technologies used in microservices architectures in so-called technology "waves." The waves address containerization, service discovery, and monitoring to continuous integration, orchestration, and serverless computing. The study presents an overview of tools that support such practices. Furthermore, the study describes four generations of the microservice architecture. Each generation is described in-depth and explained what it tried to solve from the previous generation. The study briefly mentions a significant subject within the microservices research area, which is service granularity. The authors mention the "lack of agreement on the right size of microservices.". The literature often mentions service granularity as a significant challenge. Much debate and confusion exist among the concept, which might be due to the name "microservice". Lewis and Fowler [16] suggests the name emphasizes size too much. The confusing occurs in the industry, where many project teams each follow their definitions, e.g., Amazon describes its definition according to the 'two-pizza team,' i.e., the service size should be small enough to be able to be maintained by a small team which can be fed with two pizzas. While Lewis and Fowler might be correct, it is essential to talk about service size since evidence suggests it can impact the performance of a system. Finding the right granularity is challenging, and it might be different for each situation. Mark Richards and Neil Ford [20] claim that making the service size too small results in poor performance due to additional intercommunication between services. Klock et al. [15] state that the size of service directly impacts performance and scalability.

A qualitative study by Bucchiarone [5] describes their experiences on how transforming a monolith to a microservices architecture improves scalability. The authors present the experience report through a banking application and claim that limiting a microservice to specific functionality directly benefits the system's scalability. Consequently, adding improvements or bug discoveries do not impact other services when deployed. The example might show the importance of limiting a microservice to a single business capability over coarse-grained services, and developers need to follow strict rules need to achieve loosely coupled services. Furthermore, the authors describe the benefits of a microservices architecture: scalability, loose coupling, and high coherence. However, it also presents new challenges, e.g., fault-tolerance mechanisms, concurrency handling, and increased complexity in monitoring. A more recent qualitative study by Baškarada et al. [4] presents through seventeen interviews with experienced IT architects an overview of challenges and (industry) experiences adoption and implementation of microservices. The study describes that there is much professional uncertainty related to the adoption of a microservices architecture. While the experts confirm most of the benefits described earlier, few organizations seem to shift to microservices, and according to the study, significant reservations relating to technical and organizational aspects exist. Furthermore, the authors mention that their findings may indicate that not all corporations equally benefit from the microservices architecture.

Since design has been a focal point in research, it is essential to understand why microservices came into existence to address design adequately. Heinrich et al. [12] state that the Microservices architecture serves improved development, delivery, and deployment. Chen et al. [6] mentions the advantages of microservices which are maintainability, reusability, scalability, availability, and automated deployment. Microservices try to introduce flexibility to a project, improved software development cycle, and a loosely coupled system. To achieve this, Hasselbring and Stainacker [10] propose the share-nothing approach, which means each microservice is responsible for its data. Mark Richards and Neil Ford [20] suggest that it is preferred to duplicate code over coupling to keep services independently deployable. However, Yanaga [25] as well as Lewis and Fowler [16] claim that information can be shared among microservices and does not necessarily harm coupling. These contradictions show the lack of agreement and the need for additional research.

The last decade shows many studies and books, which propose approaches to aid developers in building microservices. A book by Newman [22] describes an in-depth guide to building microservices and the possible side effects of the implementation. A recent book by Newman [19] describes an in-depth guide decomposing monolithic applications into a microservices architecture. A study by R. Chen [6] describes an approach decomposing a monolith to a microservices architecture. A tool developed by M. Gysel et al. [9] called Service Cutter tries to automate the design process. Service Cutter requires an entity relation diagram in the form of a JSON file. Service Cutter uses graph clustering algorithms in order to suggest potential microservices. Service Cutter is often cited in the literature and used to make comparison-based studies. Baresi et al. [3] suggest their algorithm and compares it with the results of Service Cutter. Another study by S. Tyszberowicz [23] does the same with their algorithm. It might suggest that Service Cutter is a successful tool to aid in building a microservices architecture.

While performance is essential in software development, faster is not by definition better. Google sees a 20% drop in traffic after a response time of 500 ms, and Amazon sees a 1% reduction in sales for every 100 ms of latency. However, both a study by Card et al. [14] and a study by Miller [18] found that users experience is affected by the response time according to different scales. The studies found that at most 200 ms, the user experiences the response as real-time, a response time between 200 ms and 1000 ms requires some feedback, and a response time between 1 s and 10 s requires a loading indicator. Therefore it could be a matter of usability design instead of performance which loses customers.

While the microservices architecture is often referred to as being cloud-native, a significant development in cloud computing is the serverless technology. With serverless technology, a developer decomposes a program into functions that live independently in the cloud. Infrastructure management, e.g., capacity provisioning, patching, and scaling, is the cloud provider's responsibility. In contrast, the cloud consumer focuses solely on writing code. Serverless services often come with autoscaling, high availability, and pay-for-value billing. A study by Baldini et al. [2] describes the serverless technology challenges and research directions. It is essential to be aware of serverless technology since it builds on top of the microservices concepts of independent and flexible programming. It might also suggest that having small services is not as bad as some researchers claim.

Chapter 3

Method

The following chapter provides a detailed description of the research methodology and methods used. Section 3.2 describes the controlled experiment, which forms the basis of the study. Subsection 3.2.6 and subsection 3.2.7 describe how data is measured and how analysis is performed. Furthermore, section 3.3 describes how the variety of systems are designed and presents an overview of diagrams that visualize the system's design.

The study considered a variety of study designs. A Survey can be used to collect data from a larger sample [8]. While it can be beneficial, it does not suit the goal of the study. Surveys describe phenomena in the real world; however, it is difficult to study specific independent variables and their effects. Possibly other factors could affect the results. The case study design [21] poses a suitable option. A case study would study a real-world application. While providing a realistic and complete view, it is difficult to isolate certain variables. Consequently, it would be too time-consuming to find if and which real-world system uses one of the three specific functional decomposition technique. Hence, the study will use a controlled experiment [24] which allows the study to do so. A controlled experiment allows for higher control, testing hypotheses, and study the effects of specific independent variables.

3.1 Systematic Literature Review

The study presents a Systematic Literature Review (SLR) in chapter 2 Related Work before the start of the experiment. The SLR utilizes a variety of academic libraries such as Scopus, ACM, and IEEE. The main keyword was "microservices" and "microservice decomposition". The keyword "microservice" resulted in 1,256 articles, and the keyword " microservice decomposition" resulted in 87 articles. The SLR assessed articles based on the publication citations and peer reviews. Below lists additional inclusion and exclusion criteria.

Inclusion criteria

- Article can be freely obtained
- Article is published in the last decade
- Article is available in English
- Article is available in full text online

Exclusion criteria

- Short papers with limited details
- Reviews
- Editorials

3.2 Controlled Experiment

The study uses a controlled experiment to test three functional decomposition techniques: Event Storming, Actor/Action and Service Cutter. The techniques are taken from the literature and specifically chosen to output a variety of different architectural designs. The study will use each treatment to implement the same use case. The CoCoMe platform described by R. Heinrich [11] will be used to form the basis of the experiment. Section 3.3 describes details about the use case. The study measures performance and scalability, and maintainability for each treatment. The experiment will result in three sets of data for three different quality metrics, subjected to statistical analysis to test the three hypotheses.

3.2.1 Validity Threats

The study design contains a variety of validity threats. Subsection 3.2.2 describes the internal validity threats and subsection 3.2.3 describes the external validity threats of the study. Additionally, the following subsections include a variety of counter measurements. The study is aware of the trade-offs between internal and external validity and designed the controlled experiment with significant control over extraneous factors. Furthermore, chapter 5 describes additional future work which can compliment the current study design and their outcomes.

3.2.2 Internal Validity

- **Threat**: Functional decomposition techniques could potentially threaten the internal validity of the study. In specific, Event Storming and Actor/Action oppose some subjectivity as it could result in different outcomes under a different researcher.
  Counter: The study follows a standardized use case – the CoCoMe framework. Due to the wide use of the use case within the literature, the resulting architectural designs can be compared to similar studies to verify outcomes. Consequently, Service Cutter does not need any human interference as it is an algorithmic functional decomposition technique.

- **Threat**: Implementation details could potentially threaten the internal validity of the study. The implementation opposes some subjectivity as it could result in different outcomes under a different researcher.
  Counter: The study keeps implementation details constant (control variable) as it is not of interest to the study. Note that the study is concerned with the differences between treatments rather than actual outcomes. A monolithic

implementation forms a single source of truth, and each treatment copies code from the monolith to keep implementations constant. A Control variable enhances the internal validity and limits the potential impact of confounding and other extraneous variables.

Given three functional decomposition techniques a, b, c, implementation details λ, and outcome α. The experiment f is denoted as

$$f(a + \lambda) = \alpha_a, f(b + \lambda) = \alpha_b, f(c + \lambda) = \alpha_c$$

by keeping λ a control variable it can be excluded from the observed difference β denoted as

$$f(a) = \beta_a, f(a) = \beta_b, f(c) = \beta_c$$

- **Threat**: Deployment design could potentially threaten the internal validity of the study.
  Counter: As with the implementation details, the study keeps the deployment constant. The study throttles initial requests with 100 ms. It is trivial to see that the initial request is a control variable itself. An initial request always occurs exactly once for any test case for any treatment. Additional requests contain an 8 ms latency since it is the mean response time after testing intercommunication within a cloud platform. A Control variable enhances the internal validity and limits the potential impact of confounding and other extraneous variables.

3.2.3 External Validity

- **Threat**: The use case could potentially threaten the external validity of the study, as it could be difficult to generalize to a larger population.
  Counter: The study follows a well-described and documented use case from the literature [11] to limit its impact. The CoCoMe framework includes enough elements to generalize to a bigger population. i.e., Web applications consisting of a GUI, additional logic, and database read/writes.

3.2.4 Independent Variables

The study identifies three independent variables. Note that the independent variables are not arbitrary. Firstly, all variables are well defined within the literature and, most importantly, output each a unique architectural design to study and compare (coarse-grained to fine-grained). It is trivial to see that the same architectural design results in the same behavior. Secondly, Service Cutter is a tool often mentioned within the literature, and in specific, its deterministic behavior makes it unique compared to all other functional decomposition techniques. The following independent variables are present:

- Service Cutter
- Actor/Actions Approach
- Event Storming

Time in seconds	User Experience	UI Solution
< 0.2	Real time	None
0.2 - 1	Minor delay	Requires some feedback
1 - 10	Significant delay	Requires a loading indicator
> 10	Unresponsive	None

Table 3.1: Response Times and User Experiences [14][18].

3.2.5 Dependent Variables

The study identifies three dependent variables. Note that the dependent variables are not arbitrary. The literature often mentions performance (issues), and it would be valuable to know if performance behaves differently among different microservice architectural designs. Consequently, the literature mentions scalability and maintainability as one of the most important advantages of microservices architecture [1][4]. It could be valuable to know whether the architectural design impacts scalability and maintainability to exploit them further. The following dependent variables are present:

- Performance
- Scalability
- Maintainability

3.2.6 Data Gathering

Performance

The study measures the performance metric in response time in milliseconds. The response time consists of Service Time + Network Latency. The study gathers 20 batches. Each batch consists of 100 samples for each functional requirement and discards the first batch i.e., The application makes use of additional optimization techniques and caching. Additionally, the study divides performance into four rankings. Table 3.1 shows the performance rankings from the studies [14] [18]. A performance measurement that falls into a certain rank is not considered significantly different from any performance measurement which falls in the same rank.

Scalability

The study is concerned with the Y-axis of the scalability cube. i.e., functional decomposition. The study measures scalability both in response time in milliseconds and throughput, number of requests per second. Consequently, the study categorizes the average response time according to the performance ranking described in table 3.1. As with performance measurements, each batch consists of 100 samples. Each batch, however, consists of multiple consecutive requests; therefore, a batch differs in size with the number of consecutive requests, e.g., 20 consecutive requests in a

batch size of 100 samples consisting of $20 * 100 = 200$ samples.

Note that the perceived time by a user is different from the mean response time. The 90th percentile is close to the perceived time of a user. Therefore the study uses the 90th percentile to rank both performance and scalability measurements.

Maintainability

The study measures the maintainability index (MI) using the Halstead Volume (H), Cyclomatic Complexity (G), Count of source lines of code (S). Additionally, Source Lines of Code (SLOC) are collected to compare each implementation with another. The formula for the maintainability index is as follow:

$$MI = max[0, 100 * \frac{171 - (5.2 * \log(H)) - (0.23 * G) - (16.2 * \log(S))}{171}]$$

JMeter

Performance tests are ran using Apache JMeter. JMeter is a tool that can mimic workload and analyze the performance of a system. JMeter is often used both within the industry and academic area. The tool is well represented throughout the literature and characterized as a reliable tool to measure performance. JMeter allows creating a test plan that defines a policy to create realistic testing scenarios. All requests need to go through the GUI service, and no API request is sent directly to a microservice. Performance tests measure response time, throughput, and error rate. Each of the treatments will go through the same set of test cases within JMeter.

3.2.7 Data Analysis

The statistical significance tests are leading when analyzing results. The user experience test in section 3.2.6 forms additional support to the statistical significance tests in the study. The performance measurements will use a non-parametric Friedman test to test the statistical significance of the treatments for H1. The alpha (p) level is 0.05. In the case H1 is rejected, a Nemenyi test is applied to find which treatment shows a statistical significance. The scalability measurements will use regression analysis. Generating a parameterized model and perform a regression slope test to test the statistical significance of the treatments for H2. The alpha (p) level is 0.05. Each treatment is measured against another, resulting in three combinations to find which treatments differ. Finally, the maintainability measurements will use a ANOVA test to test the statistical significance of the treatments for H3. The alpha (p) level is 0.05. Each treatment is measured against another, resulting in three combinations to find which treatments differ.

3.3 System

The following section provides a brief overview of the design documentation and functional requirements. The study uses the use case described in the CoCoMe framework [11]. However, slight changes may apply and are detailed below. Subsections 3.3.1 and 3.3.2 present an overview of the deployment and technologies. From subsection 3.3.3 the system is described in-depth. Subsection 3.4 presents the architectural overview for each treatment.

3.3.1 Implementation

Each implementation is written in Javascript using the Node.js library. Services are using REST APIs in order to communicate with each other. The study uses the Express library to create web applications and standardize the RESTful communication between services. The study uses MongoDB databases to store documents through the Mongoose library Mongo DB.

3.3.2 Deployment

The design of the deployment aims to increase control on the experiment and reduce several impacting factors on network latency, e.g., server location, server quality, configuration, and user location. The study deploys and tests each implementation on an isolated machine with limited resources, 1 CPU core with 1024 MB of memory, and simulates initial requests with 100 ms and additional requests with 8 ms (discussed in section 3.2.2 Internal Validity, deployment threat). The study uses PM2 as a process manager to automatically start and provision the microservices. PM2 runs Node.js applications in production mode as a single instance with the option exec-mode "fork" and default settings.

3.3.3 Purpose

The system represents a real-world scenario of a store and its supply chain. Customers can make purchases in a store. Stores can sell products as well as order products from a distributor. Managers can read reports and make changes to the store. Figure 3.1 shows the use case diagram relevant for the system.

3.3.4 Actors

The system identifies five actors:

- Customer
- Cashier
- Store Manager
- Stock Manager
- Enterprise Manager

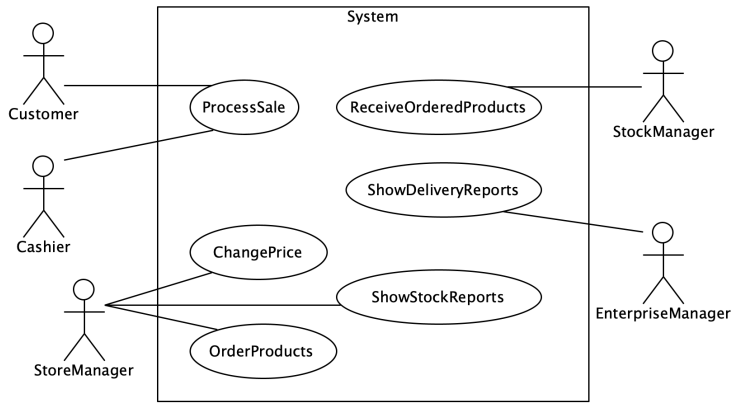

Figure 3.1: Use Case Diagram for the CoCoMe use case.

3.3.5 Requirements

Based on the use case diagram in figure 3.1 the system identifies eight functional requirements described in table 3.2.

Code	Functional Requirement
FR-01	Customer is able to make a purchase
FR-02	Customer can make an payment
FR-03	Cashier can make an sale
FR-04	Store manager can order products
FR-05	Store manager can change the price of a product
FR-06	Store manager can request an stock report of an specific store
FR-07	Stock manager can receive ordered products
FR-08	Enterprise manager can request delivery reports of an specific store

Table 3.2: System requirements for the CoCoMe use case.

3.4 Architectural Overview

The following section provides the architectural design of each of the three functional decomposition approaches. Subsection 3.4.1 provides an in-depth explanation of a monolithic application.

3.4.1 Monolith

The monolith forms the basis for the functional decomposition approaches to have similar implementations of the functional requirements mentioned in subsection 3.3.5. Code is taken from the monolith to exclude implementation differences as a potential, influential factor. Below the sequence diagrams are pictured to show communication paths within the application. Sequence diagrams are kept the same for each approach, with additional API calls instead of function calls.

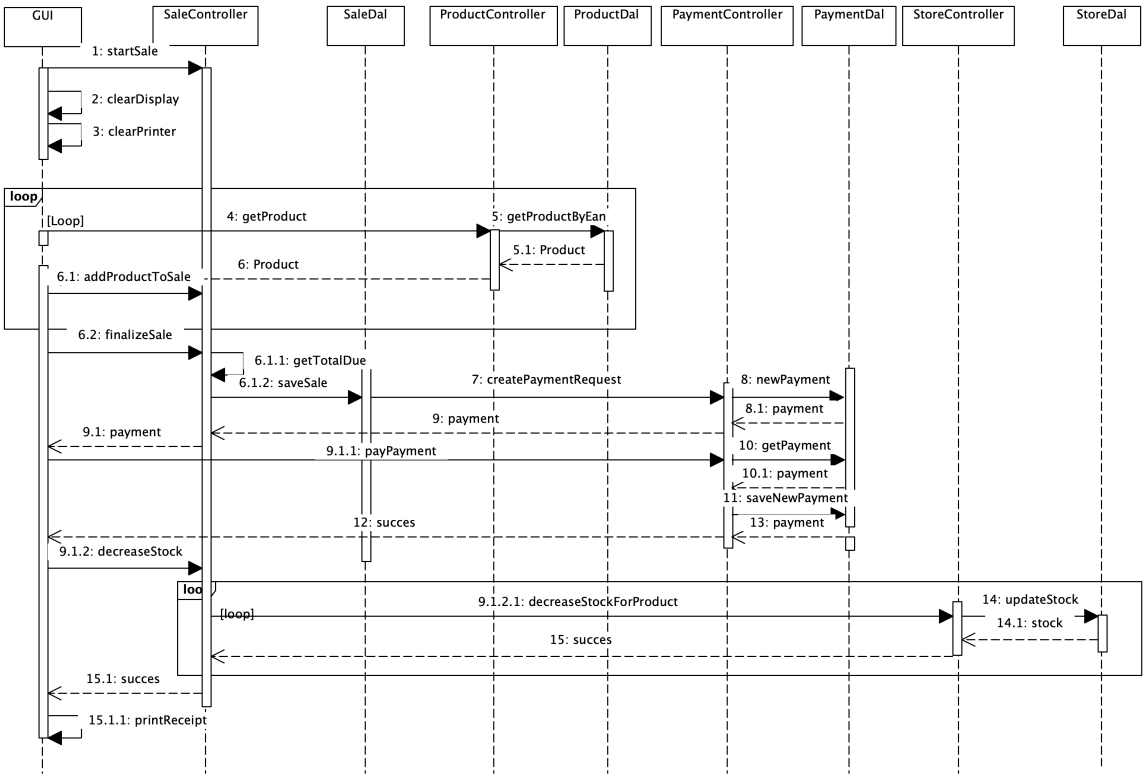

Figure 3.2: Sequence diagram of the Make Sale functional requirement.

Make Sale

Figure 3.2 shows the *MakeSale* use case. The use case consists of multiple functionalities in order to simulate a sale between customer and cashier. Make note that usually, a think time is present within certain actions, e.g., scanning a product. The actions take no think time to simplify the testing process. The test case starts by starting a sale, which resets the sale variables, printer, and display. The test case simulates the cashier scanning products and adding them to the sale. Once adding products is done, the process of finalizing the sale begins, saving the state temporarily. The system creates a payment request in the 'pending' state. The customer pays the payment, and the payment request is updated to "payed." Finally, for each product, the stock is updated, and the customer receives their receipt.

Receive Order

Figure 3.3 shows the *ReceiveOrder* use case. The stock manager receives and handles the order. The system updates the order to received. For each item, the store manager verifies the current stock and updates it accordingly.

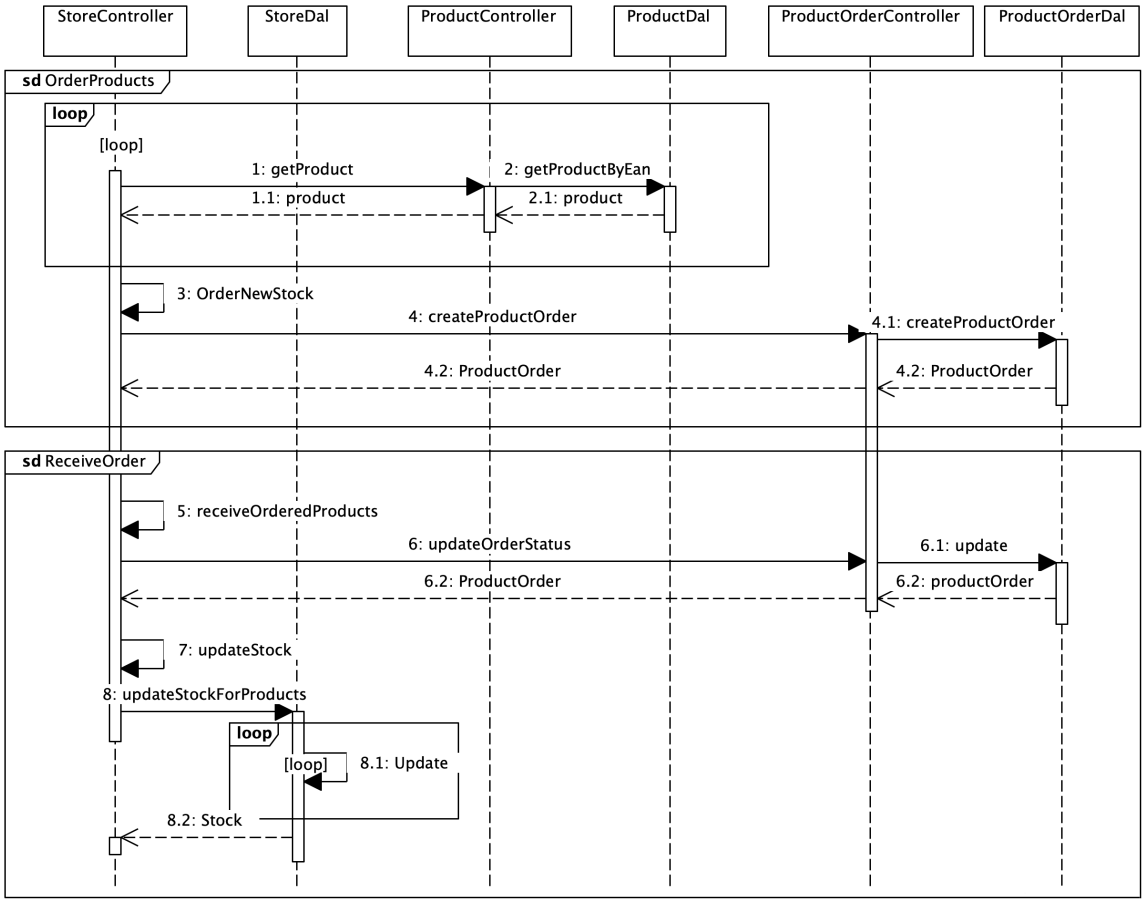

Figure 3.3: Sequence diagram of the Receive Order functional requirement.

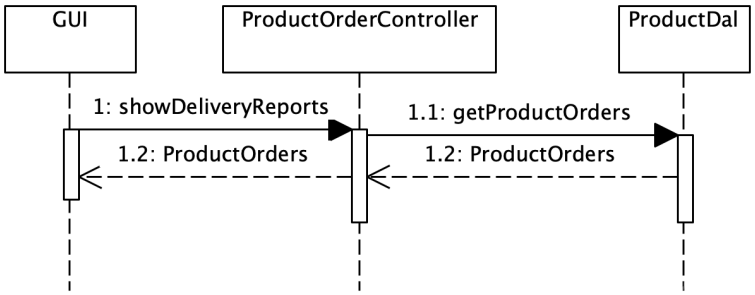

Figure 3.4: Sequence diagram of the Show Delivery Report functional requirement.

Show Delivery Reports

Figure 3.4 shows the *ShowDeliveryReport* use case. The enterprise manager can request the system for delivery reports of product orders for a store. The report consists of past and planned deliveries with their delivery times.

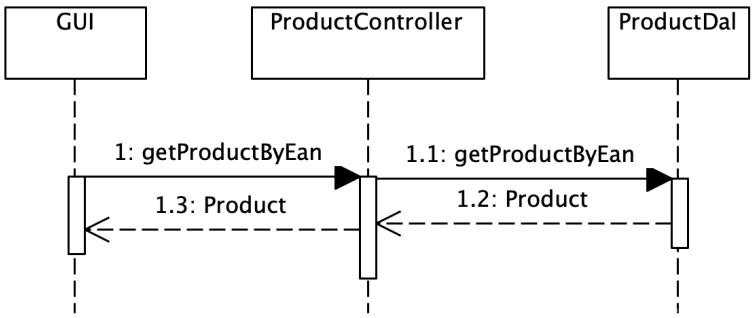

Figure 3.5: Sequence diagram of the Get Product functional requirement.

Get Product

Figure 3.5 shows the *GetProduct* use case. Each product has a unique barcode that identifies individual products. Several actors make use of the functionality. e.g., a customer could scan a product within a store to find the price, or the cashier scans products to process a sale. The implementation is rather simple, which queries the database according to the primary key and responds accordingly.

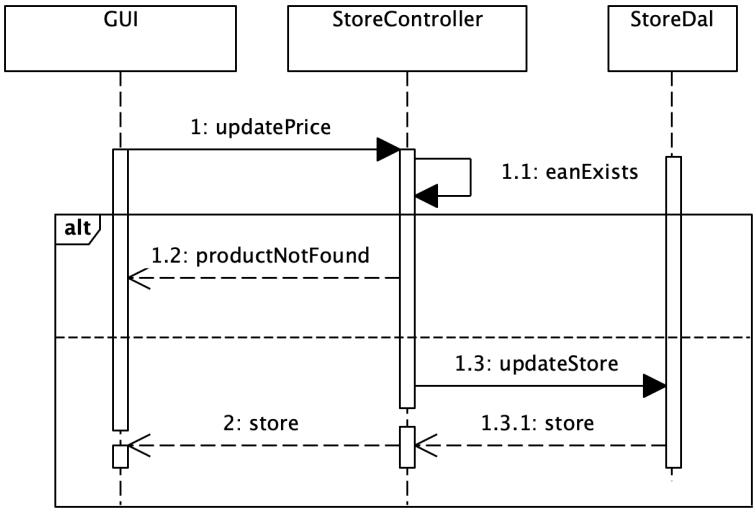

Figure 3.6: Sequence diagram of the Update Price functional requirement.

Change Price

Figure 3.6 shows the *ChangePrice* use case. The store manager can change the price of a product within a store. To update the price of a product, the store manager enters the store, product, and new price. The system checks through the specified store the existence of the product and updates accordingly.

3.4.2 Event Storming

Event Storming is an approach that aims to reduce communication between components—often resulting in merging several services into one resulting in bigger services. Using the Event Storming technique with the specifications of the CoCoMe use case results in an initial design of four microservices: Sale service, Store service, Product service, and Report service. The design is similar to [11] and [23], hence a reasonable solution for the CoCoMe use case. However, merging the report service could oppose some performance benefits while still adhering to cohesion and loose coupling. Merging the reporting service avoids the overhead of intercommunication at the cost of some code duplication, which some may suggest is a good practice [20]. The reporting functionality will be implemented in both the Sale service and Store service, making the separate Report service obsolete. The final design consists of three microservices: Sale Service, Store service, and Product service, respectively. Figure 3.7 presents the architectural design for Event Storming. Comparing the results to other studies shows that the final design is a reasonable design for the specific use case.

3.4.3 Actor/Action

The Actor/Action approach identifies typical actors and their activities and suggests building around those. Actor/Action is somewhat the opposite of Event Storming. Event Storming tries to merge services as much as possible, Actor/Action tries to separate functionality as much as possible. This can be seen in the resulting architecture in figure 3.8. Overall it shows similarities to figure 3.7,[11], and [23]. However, a

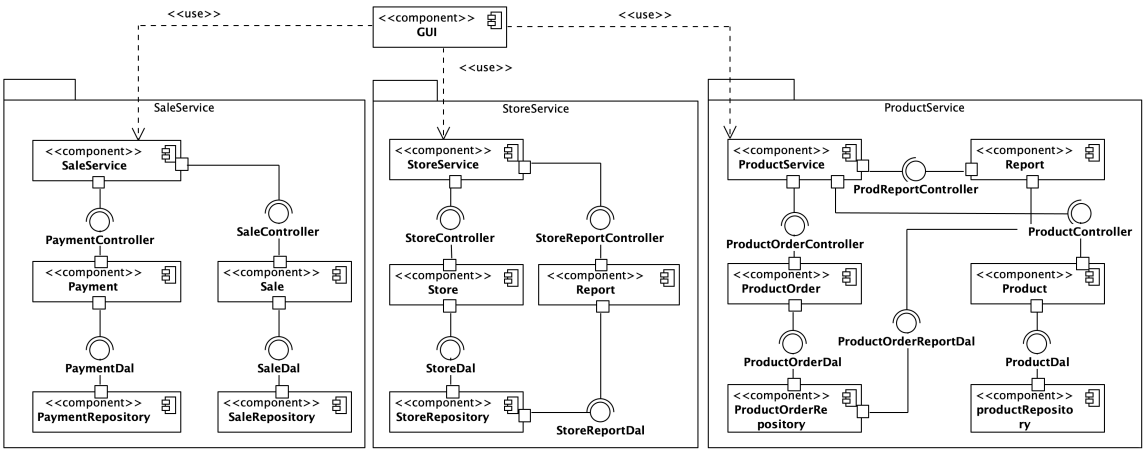

Figure 3.7: Component diagram using the Event Storming technique.

Sale service now has the payment functionality separated into its microservice. Also, the Reporting service is not included in the Sale service, as was the case with Event Storming. The same is true for the Product service, which splits into a Product service and Product Order service. Finally, the Stock service separates from the Store service. Figure 3.8 presents the architectural design for Event Storming. It is trivial to see that the architecture is more fine-grained compared to figure 3.7. The major difference to Event Storming is that Actor/Action prefers separating instead of merging services. Overall, it still shares many similarities to [11], and [23]; however, the four initial microservices are now further split into smaller microservices, potentially increasing cohesion and loose coupling.

3.4.4 Service Cutter

Service Cutter transforms an ERD into a microservices architecture. The ERD is built from the monolith application presented in [11] and described in subsection 3.4.1. Optionally an additional JSON file can be specified in order to improve potential service cuts. Therefore, Service Cutter works exceptionally well if the design of a system is already available. Service Cutter supports several clustering algorithms. The experiment uses the Markov Clustering Algorithm, which is an unsupervised cluster algorithm for graphs. Consequently, Service Cutter has a variety of additional options which could improve service cuts. The experiment uses default options. Figure 3.9 presents the initial design by Service Cutter. The design shows many similarities to [11], and [23]; however, Service Cutter chooses to separate the Product service into two services: Product service and Product Order service, which the Actor/Action architectural design also had. Furthermore, the Payment service and Sale service are separated, which differs from Event Storming but not from Actor/Action. Therefore, Service Cutter is an intermediate technique compared to Event Storming and Actor/Action and shows similarities to both designs. Figure 3.10 presents the final design.

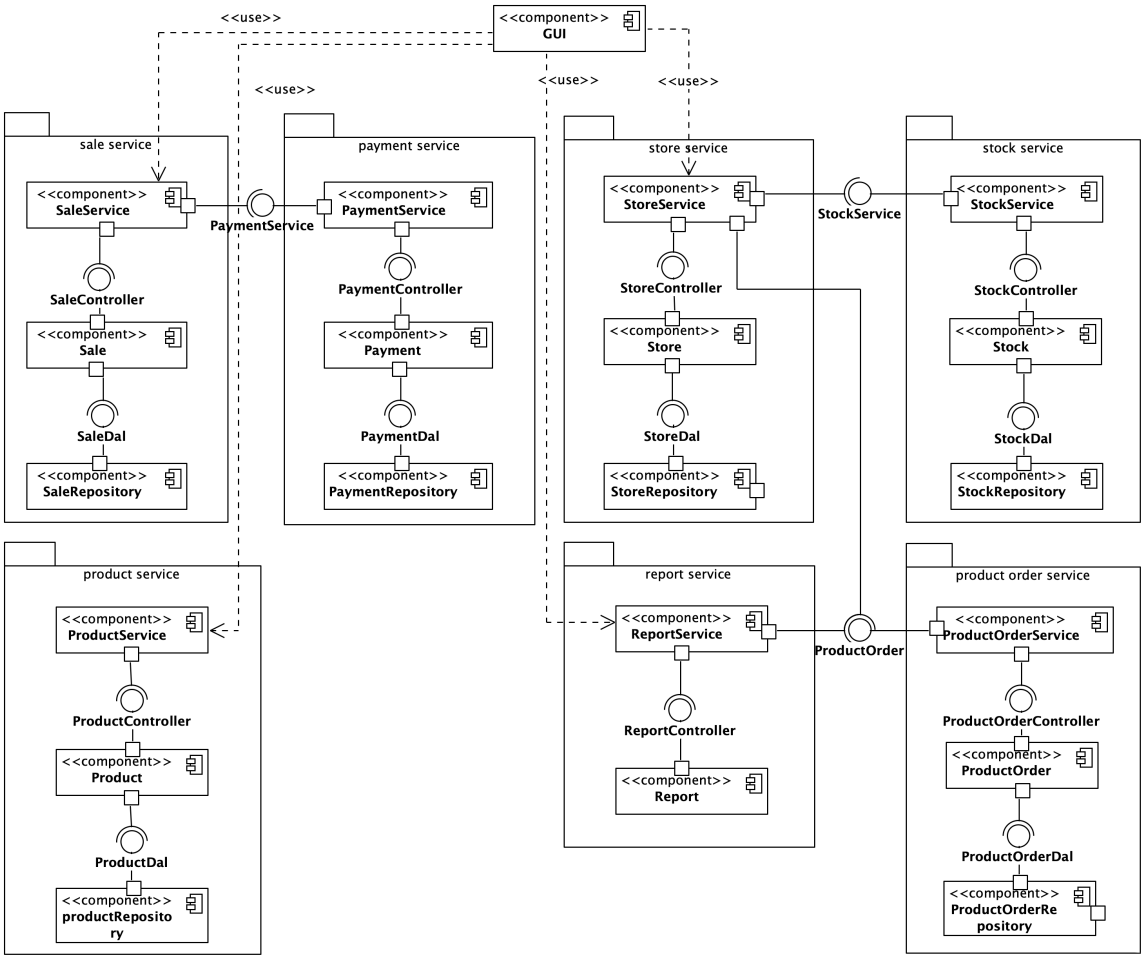

Figure 3.8: Component diagram using the Actor/Action technique.

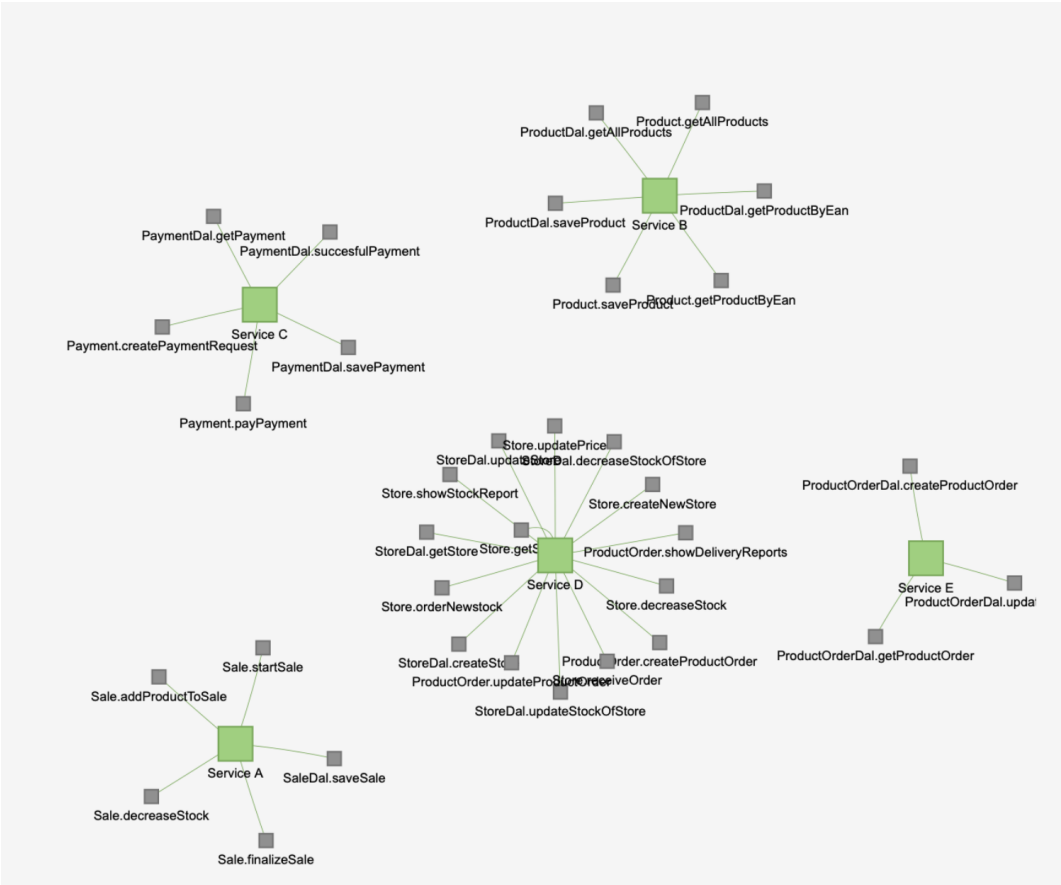

Figure 3.9: Initial suggested design by the Service Cutter tool.

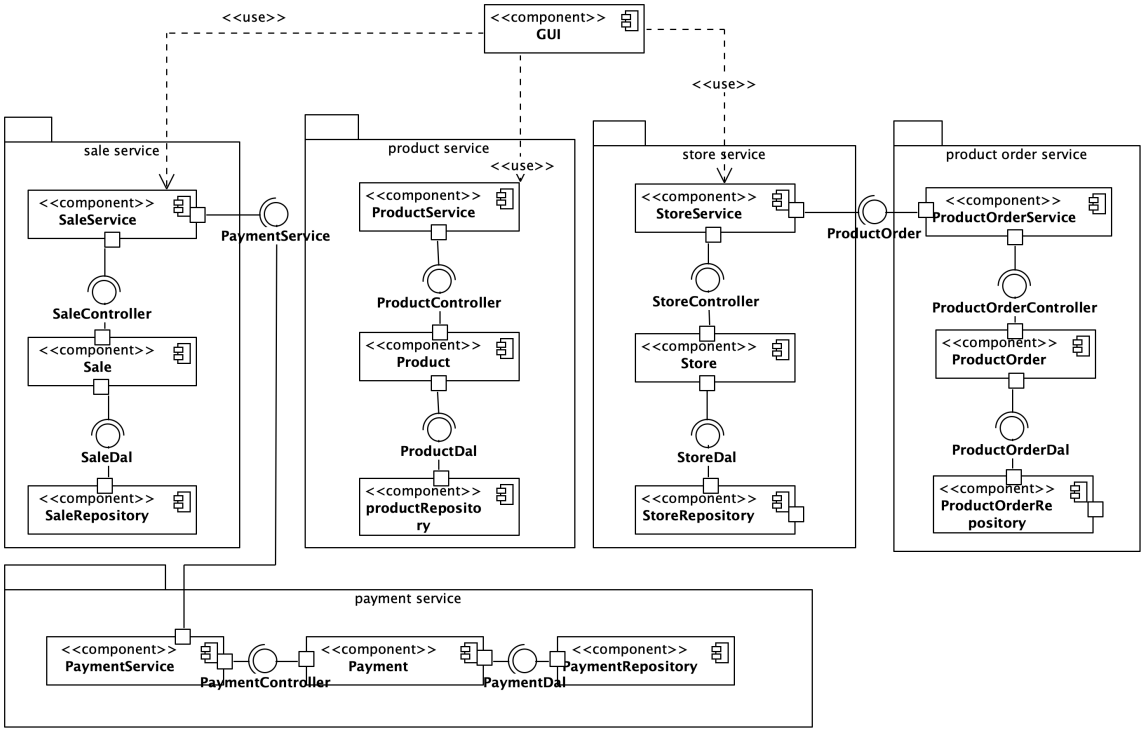

Figure 3.10: Component diagram using the Service Cutter tool.

Chapter 4

Results and Discussion

The following chapter presents the results and discussion. Section 4.1 presents the results of the conducted experiment. Possible explanations are discussed in section 4.2.

4.1 Results

Each quality metric is presenting its results in a separate subsection. Subsection 4.1.1 describes the results related to the performance, subsection 4.1.2 describes the results related to scalability and subsection 4.1.3 describes the results related to maintainability.

4.1.1 Performance

The study measures the performance in response time in milliseconds. Figure 4.1 presents the results of five test cases. A request consists of a single thread, no think time, and requests are in sequential order.

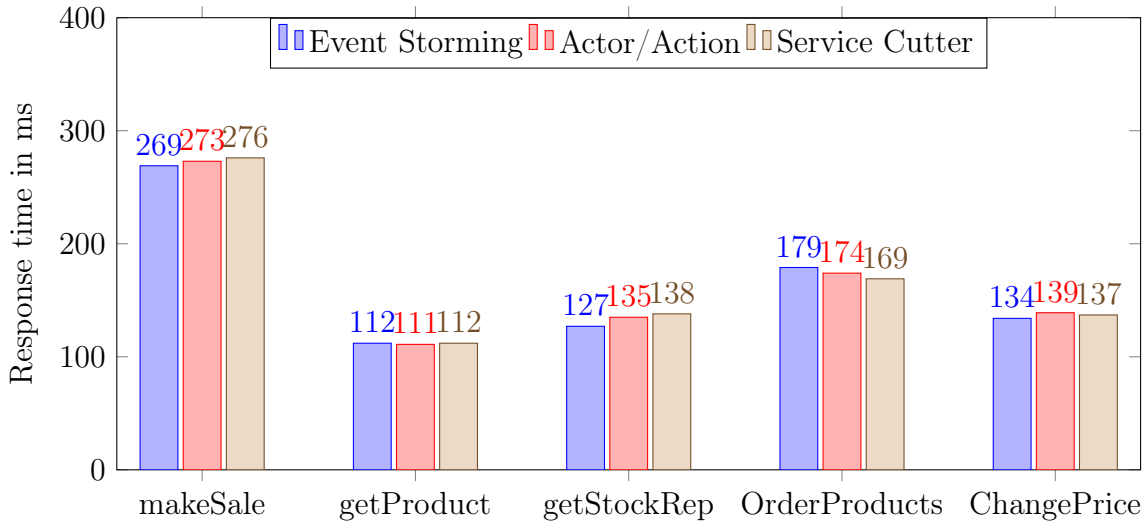

Figure 4.1: Performance: Response time in ms of the test cases.

The Friedman test shows no statistical significance. Table 4.1 presents the corresponding p-values. None of the results are significant at $p < .05$. Therefore H1

Test Case	p-value
makeSale	.1028
getProduct	.67032
getStockReport	.97531
OrderProducts	.15335
ChangePrice	.83946

Table 4.1: Results of the Friedman test.

cannot be rejected. Consequently, table 4.2 presents the ranking results according to the usability response times presented in chapter 3. The study uses the 90th percentile to rank them accordingly. Based on their ranking, none of the results show a significant difference. Therefore H1 cannot be rejected. Since both tests failed to reject H1, H1 is accepted.

Test Case	Event Storming	Actor/Action	Service Cutter
makeSale	Minor delay	Minor delay	Minor delay
getProduct	real time	real time	real time
getStockReport	real time	real time	real time
OrderProducts	real time	real time	real time
ChangePrice	real time	real time	real time

Table 4.2: Performance ranking user experience based on 90th percentile.

4.1.2 Scalability

Figure 4.2 and figure 4.4 presents the response time in milliseconds of *makeSale* and *changePrice* respectively. Figure 4.3 show the throughput in requests per seconds.

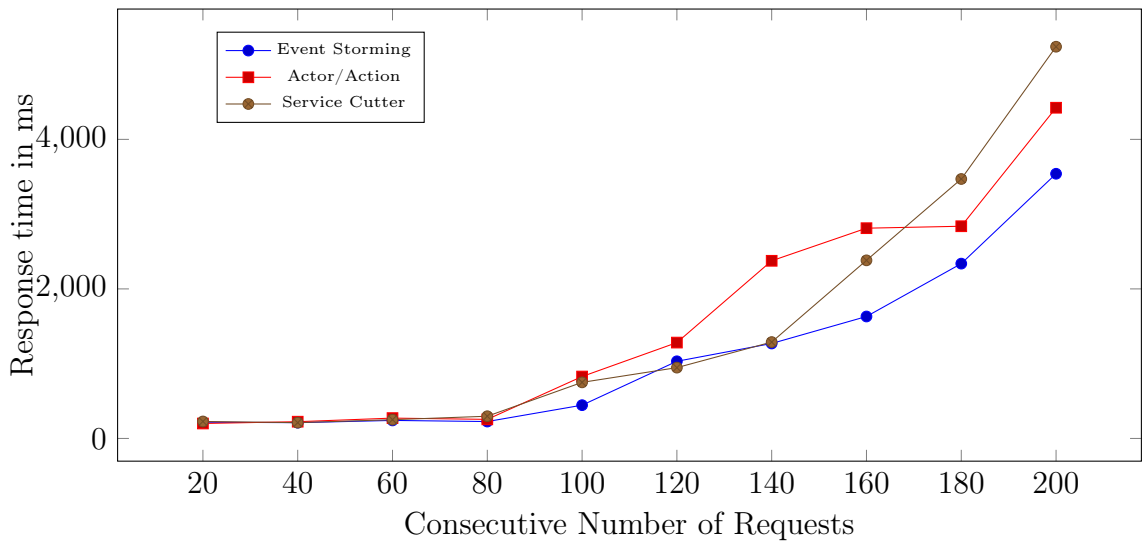

Figure 4.2: Scalability makeSale: Response time in ms and think time of 3 sec.

While *makeSale* is one of the more communication-intensive test cases, until 120 consecutive requests, each treatment could still respond to each request below the 1-second threshold. At higher load Event Storming was faster, Actor/Action was 22.18% slower than Event Storming. Service Cutter performed even worse and was 38.71% slower than Event Storming.

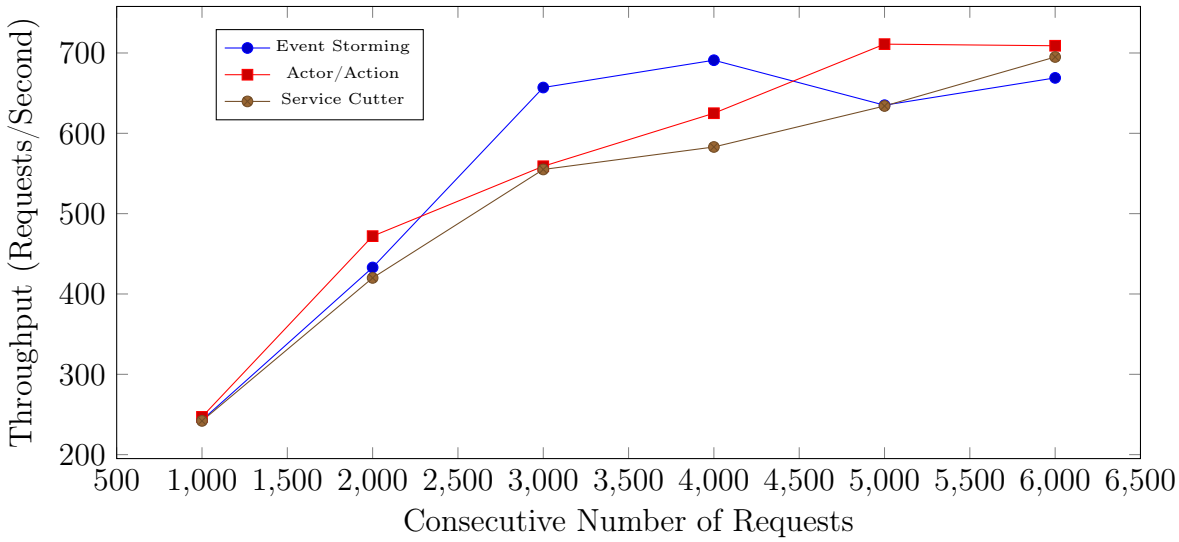

Figure 4.3: Scalability getProduct: Throughput (requests/second) and think time of 3 sec.

Until 4000 consecutive requests, Event Storming was performing better than the other treatments with higher throughput. Event storming was able to handle 16.04% more requests per second than Actor/Action and Service Cutter. However, both Actor/Action and Service Cutter had a higher throughput at a higher load. Event storming reached a maximum throughput of 691,3 requests per second; Service cutter reached a maximum throughput of 695 requests per second, and Actor/Action performed best and reached a maximum throughput of 709,1 requests per second. Actor/action was outperforming Service Cutter at the highest load tested by 2.54% and Event Storming by 5.72%.

Test Case	EventS - Actor/A	EventS - ServiceC	Actor/A -ServiceC
makeSale	.1478	.15627	.7529
getProduct	.2093	.9387	.2337
ChangePrice	.7774	.8811	.8360

Table 4.3: P-values of regression slope test on scalability results.

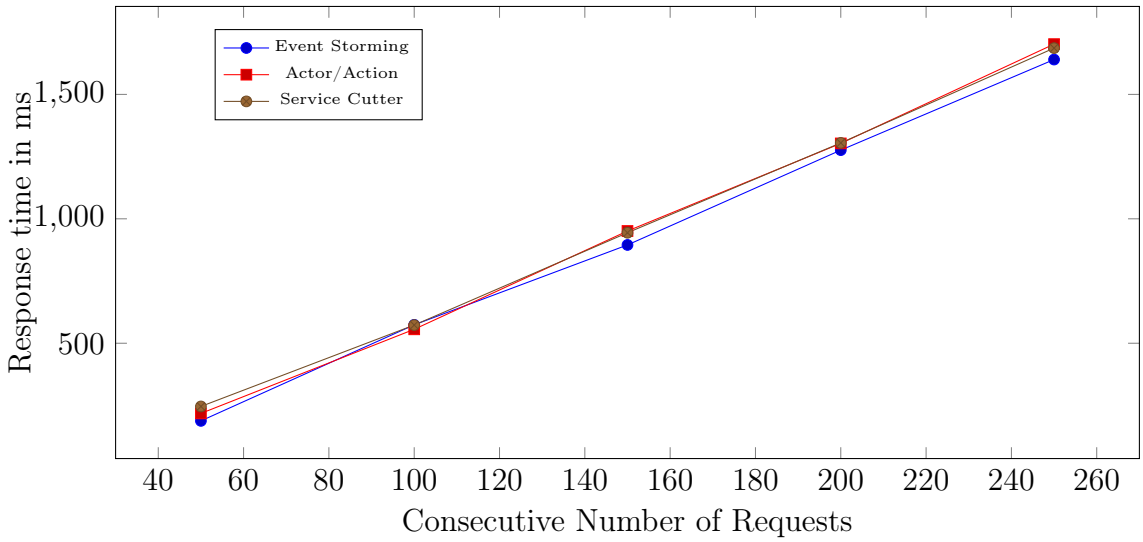

Figure 4.4: Scalability changePrice: Response time in ms and think time of 3 sec, each batch consists of a 1000 samples

Figure 4.4 presents the results for the *changePrice* test case. The treatments showed similar results. The response time increased linearly to the number of consecutive requests. Event Storming was slightly performing worse than both Service Cutter and Actor/Action by 3.71%. The study found no statistical significance using regression analysis and the regression slope tests. Table 4.3 shows the p-values, which none of the results are significant at $p < .05$. Therefore H2 cannot be rejected and H2 is accepted. Consequently, none of the ranking results show a significant difference. Therefore H1 cannot be rejected. Since both tests failed to reject H1, H1 is accepted.

4.1.3 Maintainability

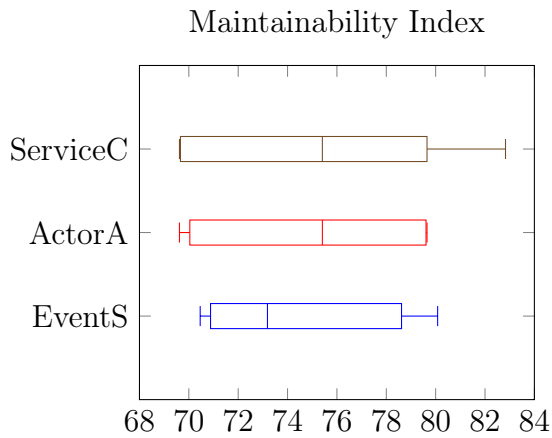

Figure 4.5: Maintainability Index.

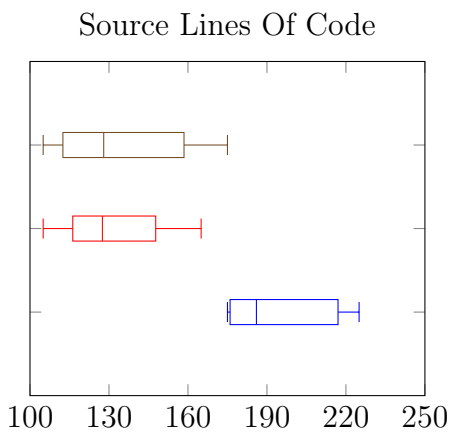

Figure 4.6: Source Lines of Code.

Figure 4.5 presents the maintainability results. Actor/Action and Service Cutter both were showing slightly better results. In figure 4.6 both Service Cutter and Actor/Action also showed a 37.32% reduction of SLOC. i.e., the microservices created through these approaches are smaller than the Event Storming. The variance (One-way ANOVA) test showed no statistical significance, resulting in an F-statistic value of 0.11855 and a P-value of 0.88924. The result is not significant at $p < .05$. Therefore H3 cannot be rejected and H3 is accepted.

4.2 Discussion

The results indicate that the service granularity does not impact performance enough to start showing significant differences. The find supports the claims by Lewis and Fowler [16], suggesting microservices emphasize size too much, and service size might not be as important as some claim it to be [19]. Although from a developer perspective and enhance reasoning, it might still be essential to form a sound definition. Describing the service size as the number of endpoints a service exposes might hold some quantitative benefits. A service exposing as little as one endpoint is the smallest a microservice can get. Consequently, exposing as few endpoints as possible could improve cohesion since by hiding information, the developer keeps changes more local. While it is still unclear the optimal number of exposing endpoints, having a quantitative definition makes it easier to reason.

While Richards [20] suggest that making the service size too small results in poor performance, the study did not found any supporting evidence. All treatments showed similar performance results. From a usability design perspective, treatments performed well within limits. Scalability did show some differences under higher load but not at a statistically significant level. The study shares some similarities with Klock et al. [15], which states that the service size directly impacts performance. There seems to be a correlation between scalability and service size but not statistically significant, and therefore, the relevance is debatable. It is reasonable to assume

that if performance issues arise, they are likely to arise in any other approach. i.e., the problem is not the design choices within a microservices architecture but the architectural choice itself. Therefore the reader should be aware that the microservice architecture is not the solution to any software product, and in some cases, other architectural approaches could be more beneficial, e.g., Monolithic architecture. Although with the rise of new communication technologies, the study expects an increase in performance and a further reduction in performance differences between approaches.

It is essential to put the performance into perspective. Further research should integrate usability design principles while studying the performance of a system. Users experience performance differently. e.g., a performance difference of 10 ms might be less relevant than a difference of 1 second. Hence it could benefit system research to integrate different fields to construct conclusions.

The study presents three functional decomposition techniques; however, it is reasonable to assume it is generalizable to other functional decomposition techniques. Especially whenever a technique shows similar architectural diagrams, they are likely to show similar behavior. Hence the study by Baresi et al. [3], and Tyszberowicz et al. [23] would result in the same kind of behavior. The find could suggest that the approach is more important in software development than the resulting architecture, i.e., An approach should be convenient, uncomplicated, and intuitive.

An important finding is that there is no significant difference between any of the treatments and Service Cutter [9], which might suggest that decision support systems can produce performing architectures compared to their human counterpart. Tools similar to Service Cutter could shift the software engineering industry to become automated and more evidence-based. It would be more convenient and repeatable to generate software architectures based on deterministic algorithms, which can be reasoned about than software architectures designed by humans heavily influenced by their experience.

Both Service Cutter and Actor/Action had the lowest lines of code per service. Knowing that the industry average is about 1 to 25 bugs per 1000 lines of code [17], smaller codebases could prevent releasing faulty code. Smaller code bases are easier to maintain and test and are therefore preferred.

4.2.1 Limitations

The study may subject to several possible limitations. The list below describes the four most likely impacting.

- **Single use case**. The study stems from a single use case, therefore making conclusive statements difficult. It is essential to conduct further research with a wide variety of use cases. Section 5.2 Future Work presents an overview of potential research directions.

- **Individual researcher**. A single researcher has conducted the study and could oppose a limitation to the research. While the researcher has been discussing progress with a superior weekly and therefore was not entirely alone, it could be valuable to include a group of researchers. Additional countermeasurements were taken and written extensively in section 3.2.1 Validity

Threats to verify any steps the researcher took. e.g., outputting architecture designs are compared to related studies to verify the design is reasonable to what other researchers have done.

- **Project Size**. The project size could potentially influence the results. i.e., Due to the project size, it could be possible that treatments are not showing a statistical significance. The project size is realistic to an experiment and partly countered through using a use case from the literature described in section 3.2.1 Validity Threats. However, it could be valuable to study a full-out production-ready system. Section 5.2 Future Work presents an overview of potential research directions.

- **Experimental Design**. While not necessarily a limitation to the study, the study had to decide between increasing internal or external validity. The current study chooses to prioritize control over variables in the experimental setup, resulting in testing the system in a lab context at the cost of generalization. It could be valuable for additional research to have less control over variables in the experimental design and increase external validity by studying the system in a real-world scenario. Section 5.2 Future Work presents an overview of potential research directions.

4.2.2 Lessons Learned

While all three approaches worked well in designing a microservices architecture, each had its advantages and disadvantages. This section presents a brief overview of the experiences while using each functional decomposition technique.

Event Storming

Event Storming resulted in coarse-grained services. Due to the coarse-grained nature of the architecture, setting up the infrastructure was significantly more manageable. Due to the limited intercommunication working on a microservice was simple. However, services could suffer the same effect of a monolith once they grow too big. Event Storming could be an excellent choice when decomposing a monolith. The resulting architecture is an intermediate step to a fine-grained microservices ecosystem. The approach especially suits well when a monolith is present, since it was easier to visualise communication streams within the system.

Actor/Action

The Actor/Action approach resulted in the most fine-grained microservices architecture. While it took considerably more time identifying all actors and actions within the system, it was also the easiest to build. Microservices were small, which resulted in easier development. Due to more and smaller services, setting up the infrastructure was presented some challenges. The Actor/Action approach can transform a monolith with ease and suits well whenever no upfront code is present. Therefore it is a good option for any situation.

Service Cutter

From all three approaches, Service Cutter was the most effortless. The resulting architecture is an intermediate step between Event Storming and Actor/Action. While Service Cutter requires an ERD, it took the smallest amount of time to use while still presenting a competing software architecture. Consequently, changing the design was uncomplicated, and due to the deterministic nature of Service Cutter, it suits well for building sustainable, repeatable software architectures. Due to its semi-automated process, the study prefers Service Cutter above Event Storming and Actor/Action.

Chapter 5

Conclusions and Future Work

5.1 Conclusion

While the study stems from an individual use case, evidence suggests that there might not be a significant difference between functional decomposition techniques. The impact on performance is minimal, suggesting that software developers have more flexibility in making design decisions. In the case performance issues arise, they will likely arise under different implementations as well. Therefore it is essential to understand that the microservices architecture does not equally benefit practitioners planning to use the architecture. Consequently, the importance should shift to the process of an approach instead of their outcome. Making the process significantly easier to use could benefit the software development process. In specific, the study prefers automated and deterministic algorithms, which could increase the quality of products and support the software development cycle.

Future studies should integrate usability design principles into their studies since they can contribute to forming a conclusion. Faster is not by definition better.

There was no statistical significance between any of the treatments and Service Cutter, suggesting that decision supports systems are improving and can compete with architectures designed by humans. While it will still take some time for algorithms to replace software architects, it is essential to create sustainable, deterministic, and repeatable software architectures. Automation could oppose a solution. It will become significantly easier for software developers to repeat what works and ignore what does not.

5.2 Future Work

It would be interesting to observe the experiment under different circumstances. A researcher could reproduce the study with various use cases, additional functional decomposition techniques, or deployment design. Consequently, future research could extend the current use case to increase complexity and come closer to a real-life scenario. Future research could also focus on a use case design methodology to investigate real-world systems. Finally, future research should focus on possible solutions with machine learning and AI for software development tools. To summarize future research:

- Replication of the current study
- Extension of the current study

- Case study of real-world microservices architecture
- Solutions with machine learning and AI to support software development

Bibliography

[1] N. Alshuqayran, N. Ali, and R. Evans, "A systematic mapping study in microservice architecture," *IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA)*, 2016. [Online]. Available: https://doi.org/10.1109/SOCA.2016.15

[2] I. Baldini, P. Castro, K. Chang, P. Cheng, S. Fink, V. Ishakian, and N. Mitchell, "Serverless computing: Current trends and open problems," *Research Advances in Cloud Computing*, 2017. [Online]. Available: https://doi.org/10.1007/978-981-10-5026-8_1

[3] L. Baresi, M. Garriga, and A. De Renzis, "Microservices identification through interface analysis," *ESOCC 2017: Service-Oriented and Cloud Computing*, vol. 10465, pp. 19–13, 2017. [Online]. Available: https://doi.org/10.1007/978-3-319-67262-5_2

[4] S. Baskarada, V. Nguyen, and A. Koronios, "Architecting microservices: Practical opportunities and challenges," *Journal of Computer Information Systems*, vol. 60, no. 5, pp. 428–436, 2020.

[5] A. Bucchiarone, N. Dragoni, and Dustdar, "From monolithic to microservices: An experience report from the banking domain," *IEEE Software*, vol. 35, no. 3, pp. 50–55, May 2018. [Online]. Available: https://ieeexplore-ieee-org.miman.bib.bth.se/document/8354415

[6] R. Chen, L. Shanshan, and L. Zheng, "From monolith to microservices: A dataflow-driven approach," *2017 24th Asia-Pacific Software Engineering Conference (APSEC)*, 2017. [Online]. Available: https://doi.org/10.1109/APSEC.2017.53

[7] P. Di Francesco, P. Lago, and I. Malavolta, "Research on architecting microservices: Trends, focus, and potential for industrial adoption," *IEEE International Conference on Software Architecture (ICSA)*, 2017. [Online]. Available: https://doi.org/10.1109/ICSA.2017.24

[8] A. N. Ghazi, K. Petersen, S. S. V. R. Reddy, and H. Nekkanti, "Survey research in software engineering: Problems and mitigation strategies," *IEEE Access*, vol. 7, pp. 24 703–24 718, 2019.

[9] M. Gysel, L. Kölbener, W. Giersche, and O. Zimmermann, "Service cutter: A systematic approach to service decomposition," *ESOCC 2016: Service-Oriented and Cloud Computing*, pp. 185–200, 2016. [Online]. Available: https://doi.org/10.1007/978-3-319-44482-6_12

[10] W. Hasselbring and G. Steinacker, "Microservice architectures for scalability,

agility and reliability in e-commerce," *ESOCC 2016: Service-Oriented and Cloud Computing*, pp. 185–200, 2017. [Online]. Available: https://doi.org/10.1109/ICSAW.2017.1

[11] R. Heinrich, S. Koch, and R. Reussner, "The CoCoME platform for collaborative empirical research on information system evolution," 2018. [Online]. Available: https://publikationen.bibliothek.kit.edu/1000085350

[12] R. Heinrich, A. van Hoorn, H. Knoche, F. Li, L. E. Lwakatare, C. Pahl, S. Schulte, and J. Wettinger, "Performance engineering for microservices: Research challenges and directions," *ICPE '17 Companion: Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion*, pp. 223–226, Apr. 2017. [Online]. Available: https://dl.acm.org/doi/10.1145/3053600.3053653

[13] P. Jamshidi, C. Pahl, N. C. Mendonça, J. Lewis, and S. Tilkov, "Microservices: The journey so far and challenges ahead," *IEEE Software*, vol. 35, no. 3, pp. 24–45, May 2018. [Online]. Available: https://ieeexplore-ieee-org.miman.bib.bth.se/document/8354433

[14] S. K. Card, G. G. Robertson, and J. D. Mackinglay, "The information visualizer, an information workspace," *SIGCHI Conference on Human Factors in Computing Systems*, pp. 181–186, 1991. [Online]. Available: https://doi.org/10.1145/108844.108874

[15] S. Klock, J. M. E. M. van der Werf, J. P. Guelen, and S. Jansen, "Workload-based clustering of coherent feature sets in microservice architectures," *IEEE International Conference on Software Architecture (ICSA)*, 2017. [Online]. Available: https://doi.org/10.1109/ICSA.2017.38

[16] J. Lewis and M. Fowler, "Microservices," 2014-03. [Online]. Available: https://martinfowler.com/articles/microservices.html

[17] S. McConnell, *Code Complete, A Practical Handbook of Software Construction, Second Edition.* Microsoft Press, 2004.

[18] R. B. Miller, "Response time in man-computer conversational transactions," *fall joint computer conference, part I*, pp. 267–277, Dec. 1968. [Online]. Available: https://doi.org/10.1145/1476589.1476628

[19] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*, 2nd ed. O'Reilly Media, Inc, 2019.

[20] M. Richards and N. Ford, *Fundamental of Software Archtectures: an engineering approach.* O'Reilly Media, Inc, 2020.

[21] P. Runeson, M. Host, A. Rainer, and B. Regnell, *Case study research in software engineering: Guidelines and examples.* John Wiley & Sons, 2012.

[22] N. Sam, *Building Microservices: Designing fine-grained systems*, 4th ed. O'Reilly Media, Inc, 2016.

[23] S. Tyszberowicz, R. Heinrich, B. Liu, and Z. Liu, "Identifying microservices using functional decomposition," *SETTA 2018: Dependable Software Engineering. Theories, Tools, and Applications*, vol. 10998, pp. 50–65, Aug. 2018. [Online]. Available: https://doi.org/10.1007/978-3-319-99933-3_4

[24] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in software engineering*. Springer Science & Business Media, 2012.

[25] E. Yanaga, *Migrating to Microservice Databases*, 1st ed. O'Reilly Media, Inc, 2017.

Appendix A

Supplemental Information

Test Case makeSale - Response Time in MS			
Batch	**Event Storming**	**Actor/Action**	**Service Cutter**
1	269	226	230
2	208	230	208
3	257	225	223
4	211	233	218
5	214	223	220
6	211	222	218
7	226	223	214
8	209	232	213
9	217	222	211
10	217	215	210
Std	55,39	65,59	60,34
90% Percentile	269	273	276
Mean	243	250	241

Table A.1: Test results of test case makeSale

Test Case getProduct - Response Time in MS			
Batch	**Event Storming**	**Actor/Action**	**Service Cutter**
1	107	109	107
2	106	108	105
3	106	104	105
4	106	107	105
5	108	105	105
6	106	109	107
7	112	104	107
8	105	107	103
9	106	106	107
10	105	107	108
Std	11,58	13,04	13,25
90% Percentile	112	111	112
Mean	110	109	110

Table A.2: Test results of test case getProduct

Test Case getStockReport - Response Time in MS			
Batch	**Event Storming**	**Actor/Action**	**Service Cutter**
1	132	113	132
2	120	119	107
3	122	114	121
4	125	118	107
5	110	113	113
6	108	117	118
7	120	116	121
8	109	111	110
9	112	112	109
10	112	113	112
Std	13,40	21,49	15,76
90% Percentile	127	135	138
Mean	120	123	119

Table A.3: Test results of test case getStockReport

Test Case orderProducts - Response Time in MS			
Batch	**Event Storming**	**Actor/Action**	**Service Cutter**
1	150	174	169
2	160	167	144
3	145	162	151
4	156	150	160
5	153	160	150
6	145	157	144
7	160	161	159
8	152	158	143
9	146	145	146
10	150	151	157
Std	28,43	34,36	33,80
90% Percentile	179	174	169
Mean	161	169	161

Table A.4: Test results of test case orderProducts

Test Case changePrice - Response Time in MS			
Batch	**Event Storming**	**Actor/Action**	**Service Cutter**
1	119	119	118
2	128	119	115
3	125	116	114
4	119	117	116
5	114	122	122
6	124	118	137
7	113	124	114
8	120	116	126
9	114	123	112
10	117	120	126
Std	21,08	22,66	21,34
90% Percentile	134	139	137
Mean	127	128	128

Table A.5: Test results of test case changePrice

Calculations	**EV_ST**	**AC_AC**	**SE_CU**	
Slope =	16,8075758	22,9763636	24,7724242	
n =	10	10	10	
SE(reg) =	497,232215	530,45235	835,869981	
SE(slope) =	2,73717326	2,92004408	4,60131281	
Difference =		6,16878788	7,96484848	1,79606061
SE(difference) =		4,00234617	5,3538955	5,44965476
t-stat =		1,54129293	1,48767351	0,32957328
df =		16	16	6
p =		0,14278933	0,15627935	0,75292397

Table A.6: Scalability test results of test case makeSale

Calculations	**EV_ST**	**AC_AC**	**SE_CU**	
Slope =	0,79111429	0,88282857	0,83771429	
n =	6	6	6	
SE(reg) =	112,589599	67,3509989	57,043008	
SE(slope) =	0,26914062	0,16099968	0,13635887	
Difference =		0,09171429	0,0466	0,04511429
SE(difference) =		0,31362011	0,30171247	0,21098493
t-stat =		0,29243752	0,15445169	0,21382706
df =		8	8	8
p =		0,7773931	0,88107905	0,83603105

Table A.7: Scalability test results of test case getProduct

Calculations	**EV_ST**	**AC_AC**	**SE_CU**	
Slope =	7,212	7,43	7,224	
n =	5	5	5	
SE(reg) =	16,6493243	17,9953698	16,7769683	
SE(slope) =	0,10529957	0,11381271	0,10610686	
Difference =		0,218	0,012	0,206
SE(difference) =		0,15505268	0,14948802	0,15560206
t-stat =		1,40597377	0,08027399	1,32388996
df =		6	6	6
p =		0,20934742	0,9386299	0,23374062

Table A.8: Scalability test results of test case changePrice

Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden